Tarkov Weapon Mod Optimizer

Automated Build Optimization using Constraint Programming

Imposs1ble嗨

September 29, 2026

The Problem: Complexity in Escape from Tarkov

Escape from Tarkov features one of the most complex weapon modification systems in gaming.

Challenges for New Players:

- **Overwhelming scale:** 2,500+ mods, each affecting recoil, ergonomics, weight, accuracy
- **Nested compatibility:** Mods attach to mods (rail $\rightarrow$ mount $\rightarrow$ sight)
- **Combinatorial explosion:** A single AR-15 has 50+ slots, millions of valid builds
- **Non-obvious trade-offs:** Is a 50k grip worth it over a 5k one?
- **Dynamic availability:** Prices change; items locked behind trader levels, quests, and flea market access

Result: Players spend hours theory-crafting or blindly copy "meta builds" without understanding trade-offs.

The Solution: Automated Optimization

Project Goal: Create a tool that mathematically guarantees the "best" weapon build for a given set of constraints.

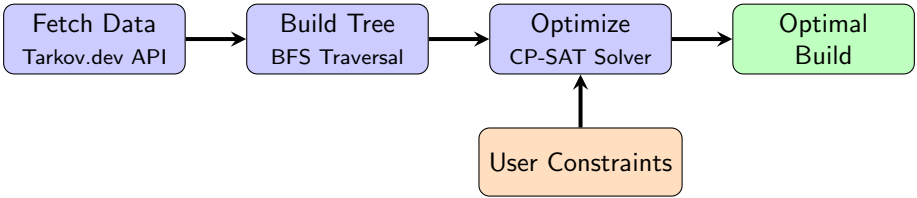
Use Cases:

- *"I have 200k roubles—what's the best M4 I can build?"*
- *"What's the absolute lowest recoil AK, money no object?"*
- *"I'm level 15 with LL2 traders—what can I actually buy?"*
- *"Is spending 100k more actually worth it?"*
- *"Show me all viable builds on a budget curve"*

Key Capabilities:

- **Multi-objective**: Balance recoil, ergonomics, and price
- **Budget-aware**: Hard limit or soft penalty on cost
- **Trader filtering**: Respect loyalty levels (LL1–LL4)
- **Flea market toggle**: Include or exclude market prices
- **Pareto frontier**: Visualize price vs. performance trade-offs

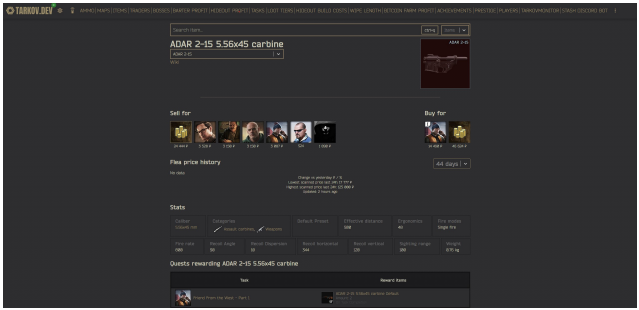
Pipeline Overview



Fetching Data

Tarkov.dev GraphQL API

- Queries all guns, mods, and compatibility rules
- **Up-to-date prices** from traders and flea market
- Trader loyalty level requirements
- Real-time flea market prices
- Local caching for speed



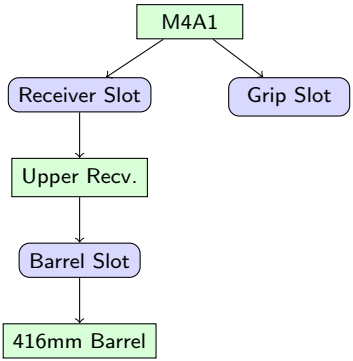
Building the Compatibility Tree

BFS Process:

1. Start from the base weapon's slots
2. For each slot, find all allowed items
3. Add those items to the queue
4. For each item, discover *its* slots
5. Repeat until no new items found

Data Structures Built:

- `slot_items`: Slot $\rightarrow$ [compatible items]
- `slot_owner`: Slot $\rightarrow$ parent item
- `item_to_slots`: Item $\rightarrow$ [owned slots]



Items own slots; slots contain items.

Introduction to CP-SAT Solver

CP-SAT = **C**onstraint **P**rogramming + **SAT** (Boolean Satisfiability)

What is it?

A solver from Google's OR-Tools library that finds optimal solutions to combinatorial problems by combining:

- **Constraint Programming**: Express complex rules declaratively
- **SAT Solving**: Efficient Boolean logic reasoning
- **Linear Programming**: Optimize numerical objectives

Why CP-SAT for this problem?

- Handles Boolean decisions naturally (select mod or not)
- Scales to thousands of variables and constraints
- Guarantees optimal solution (not just "good enough")
- Solves in milliseconds for typical weapon builds

Applying CP-SAT to Weapon Builds

Core Idea: Define what a valid build looks like, let the solver find the best one.

The Three Components

1. **Variables**: Boolean X_i for each reachable mod ($X_i = 1$ if selected)
2. **Constraints**: Compatibility rules, budget limits, required slots
3. **Objective**: Maximize ergonomics, minimize recoil and price

The solver explores millions of mod combinations, pruning invalid builds instantly, and returns the mathematically optimal configuration.

Constraint 1: Slot Mutex (One Item Per Slot)

Rule: Each slot can hold at most one item.

Formula

For each slot s with candidate items $\{i_1, i_2, ..., i_n\}$:

$$\sum_{i \in \text{slot}_s} X_i \leq 1$$

where $X_i = 1$ if item i is selected, 0 otherwise.

Example: A pistol grip slot can have a Zenit RK-3 *or* an Ergo PSG-1, but not both simultaneously.

Constraint 2: Parent Dependency

Rule: A mod can only be selected if its parent slot owner is also selected.

Formula

If item c (child) fits in a slot owned by item p (parent):

$$X_c \leq X_p$$

A child cannot be selected without its parent.

Example: You cannot attach a foregrip to a handguard unless that handguard is installed on the weapon.

$$\textit{Gun} \rightarrow \textit{Handguard} \rightarrow \textit{Foregrip}$$

Constraint 3: Conflicting Items

Rule: Some items cannot be used together (API-defined conflicts).

Formula

For each conflict pair (a, b) from the API:

$$X_a + X_b \leq 1$$

At most one of the conflicting items can be selected.

Example: Certain dust covers conflict with specific charging handles due to physical interference—the game prevents mounting both.

Constraint 4: Required Slots

Rule: Certain slots *must* be filled for a functional weapon.

Formula

For each required slot s:

$$\sum_{i \in \text{slot}_s} X_i \geq 1$$

At least one compatible item must fill the slot.

Vital Slots (AR-15 platform):

- Barrel, Gas Block, Pistol Grip, Charging Handle, Receiver, Handguard

Without these, the weapon cannot function in-game.

Constraint 5: Budget Limit

Rule: Total build cost must not exceed the player's budget.

Formula

$$\sum_i \text{Price}_i \cdot X_i \leq \text{Budget}$$

The sum of all selected item prices must stay within budget.

Smart Pricing:

- Considers trader loyalty levels (LL1–LL4)
- Checks flea market availability
- Automatically finds the cheapest source for each item

The Objective Function

Goal: Find the build that best matches the player's priorities.

Weighted Objective (Maximize)

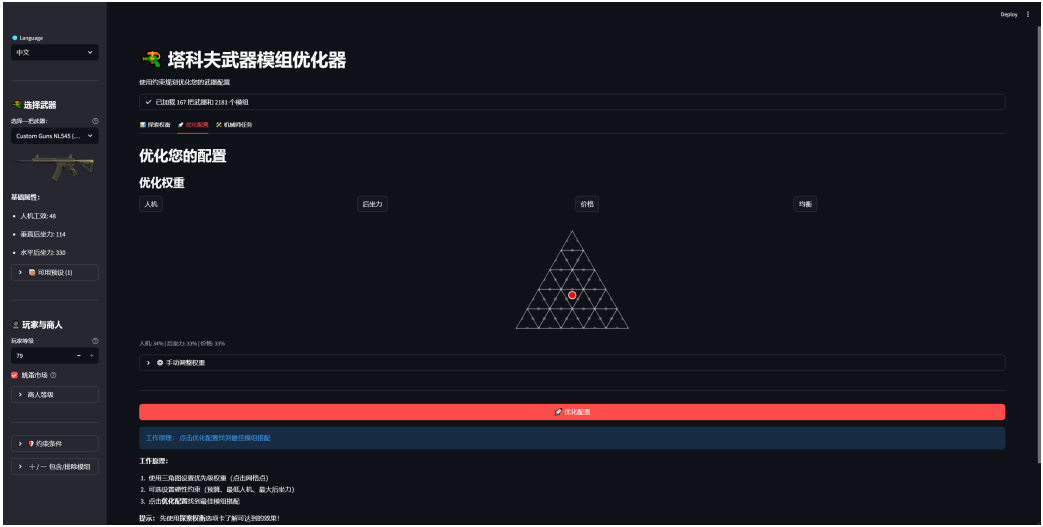
$$\text{Score} = W_E \cdot \text{Ergo} - W_R \cdot \text{Recoil} - W_P \cdot \text{Price}$$

- W_E: Ergonomics weight (higher = faster ADS, less stamina drain)
- W_R: Recoil weight (lower = more controllable)
- W_P: Price weight (lower = cheaper build)

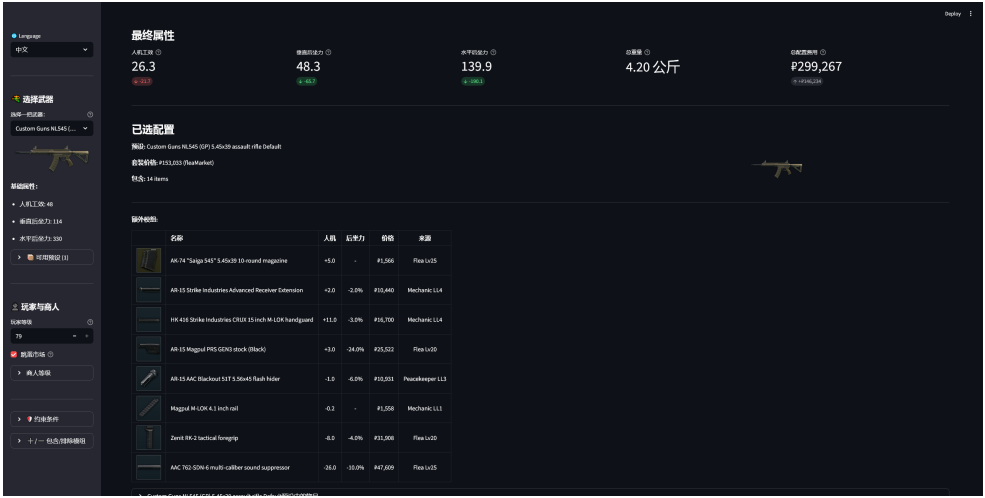
Example Profiles:

- *Meta Build*: $W_R = 100, W_E = 0.5, W_P = 0$
- *Budget Build*: $W_R = 1, W_E = 1, W_P = 0.5$

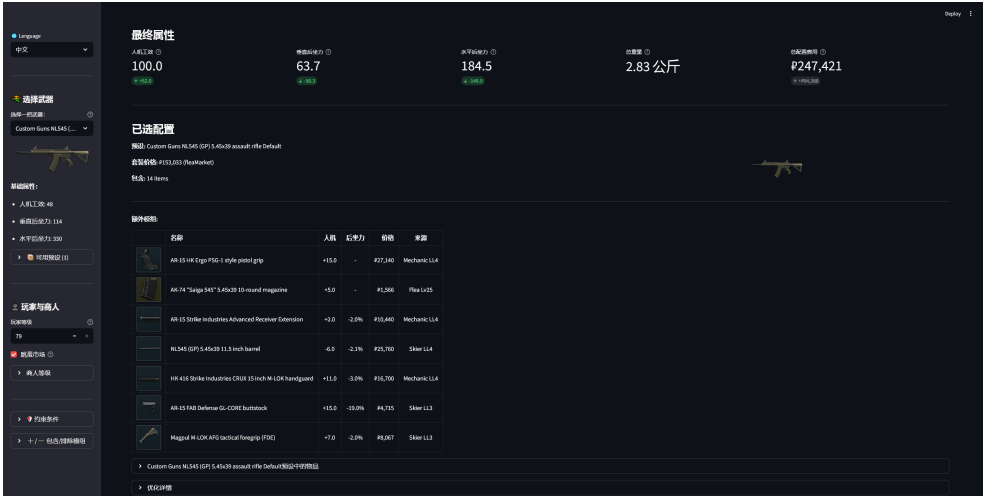
User Interface: Build Optimization



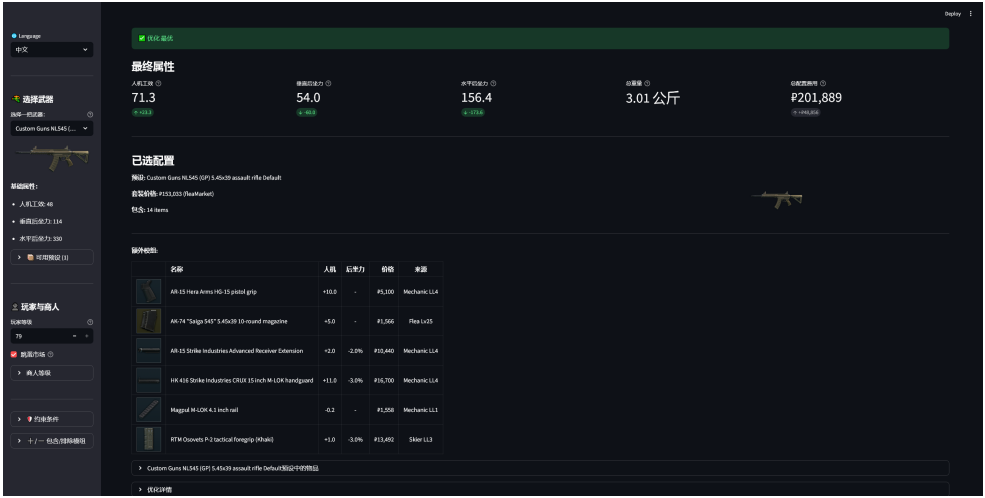
Optimization Results: Min Recoil



Optimization Results: Max Ergo



Optimization Results: Balanced Build



The Limitation of Linear Weighted Sum

Traditional Formulation: max $W_E \cdot \text{Ergo} - W_R \cdot \text{Recoil} - W_P \cdot \text{Price}$

The Non-Convexity Problem:

- Linear weighted sum sweeps **straight hyperplanes** across the objective space.
- In Tarkov, mod compatibility creates **non-convex indentations** (e.g., discrete trade-offs between heavy suppressors vs. lightweight compensators).
- **Mathematical Blind Spot:** Any point located inside a non-convex pocket **cannot be found by linear sums** under *any* choice of weights (W_E, W_R, W_P)!

Empirical Finding (Tarkov Datasets)

Across common benchmark weapons (M4A1, AK-74N, MP7A2):

- **45%–61%** of all Pareto-optimal builds lie in non-convex pockets!
- Linear weights jump directly from extreme-recoil to extreme-ergo, missing the best **"Sweet Spot"** builds.

The 3D Ideal Reference Point

Idea: Instead of arbitrary linear slopes, measure distance to a **Utopian Ideal Target**.

1. Utopia Ideal Point ($\mathbf{z}^*$):

- $z_E^* = \max_{\mathbf{x}} \text{Ergo}(\mathbf{x})$ (Best achievable Ergo)
- $z_R^* = \min_{\mathbf{x}} \text{Recoil}(\mathbf{x})$ (Best achievable Recoil)
- $z_P^* = \min_{\mathbf{x}} \text{Price}(\mathbf{x})$ (Cheapest viable build)

Computed instantly via 3 single-objective solves.

2. Normalized Gap to Target (g_i): For each objective $i \in \{E, R, P\}$:

$$g_i(\mathbf{x}) = \frac{|f_i(\mathbf{x}) - z_i^*|}{\text{Range}_i} \in [0, 1]$$

- $g_i = 0$: Reaches the theoretical best.
- Dimensionless and scale-invariant across Ergo, Recoil %, and Roubles ().

Augmented Tchebycheff Scalarization

Mathematical Formulation (Sweet Spot Mode):

$$\min_{\mathbf{x},\alpha} \left(\alpha + \rho \sum_{i \in \{E,R,P\}} \lambda_i \cdot g_i(\mathbf{x}) \right)$$

$$\text{subject to:} \quad \lambda_i \cdot g_i(\mathbf{x}) \leq \alpha, \quad \forall i \in \{E, R, P\}$$

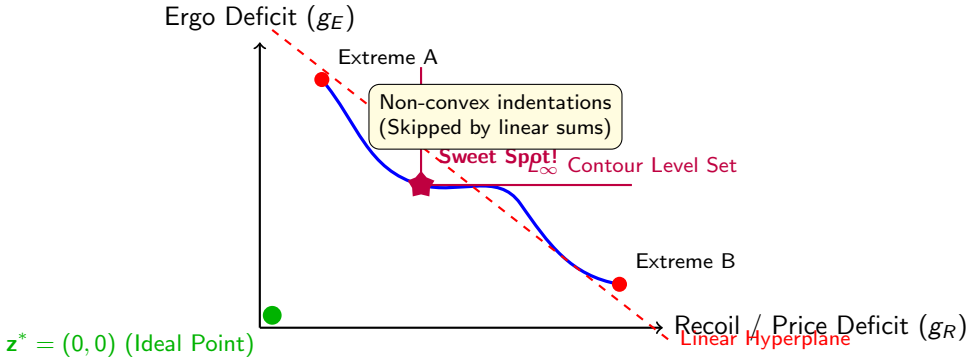
Why This Works:

- $\alpha = \max_i(\lambda_i g_i(\mathbf{x}))$ minimizes the **worst-case deficit**.
- Pulls the build uniformly towards the Ideal Point from all directions.
- Small augmentation term $\rho \sum \lambda_i g_i$ ($\rho = 10^{-4}$) guarantees **strict Pareto optimality** (eliminates flat weak optima).

MILP Integration Advantages:

- Requires only **1 extra continuous variable** (α).
- **Zero binary variables added** $\implies$ solve time remains < 15ms.
- Discovers non-convex sweet spots with 100% mathematical proof of optimality.

Geometric Intuition: Hyperplane vs. L_∞ Contour



Key Takeaway: Straight lines bridge across concave regions, while L-shaped Chebyshev contours penetrate deep into the "knee" to find balanced builds.

In-Game ADS Physics & The Overswing Problem

The Real-World Tarkov Combat Dilemma:

- High Ergonomics speeds up Aim-Down-Sights (ADS) transition and reduces arm stamina drain.
- However, heavy weapon weight introduces **rotational momentum and inertia**.

What is Sight Overswing?

When quickly snapping to aim, a weapon whose weight exceeds its ergonomic handling limit will **sway past the target reticle** before settling.

- Causes missed first shots in high-tempo CQB.
- A 100-ergo, 7.5 kg build overswings violently.
- A 65-ergo, 3.2 kg build snaps instantly and

"Raw Ergonomics is misleading without accounting for weapon weight."

Community Research:
SpaceMonkey37 & EFTForge established the empirical relationship between Ergonomics, Weight, and Overswing.

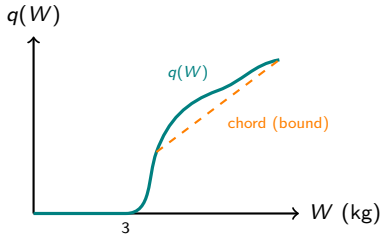
TrueErgo: The Piecewise TED Metric

Current model (v2.13): ergonomics minus a weight penalty that is free up to 3 kg and grows steeply after.

$$\mathrm{TED} = E_{\mathrm{eff}} - q(W), \qquad q(W) = \begin{cases} 0 & W \le 3\,\mathrm{kg} \\ 100 - \dfrac{300}{W} & W > 3\,\mathrm{kg} \end{cases}$$

$$E_{\mathrm{eff}} = \mathrm{clamp}_{[0,100]}\big(E_{\mathrm{raw}} \cdot (1 + b)\big)$$

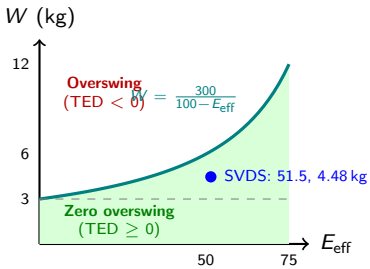
- TED ≥ 0: no overswing. This is exactly the **Prevent Overswing** constraint.
- The penalty is **concave** above 3 kg, so a chord through it *underestimates* q and gives a valid upper bound on TED.
- The solver reports objective bounds and a gap; "Feasible" means the time limit hit before the gap closed.
- Example (SVDS): 51.5 ergo, 4.48 kg $\Rightarrow$ TED $\approx +18$.



Hard Constraint: Prevent Overswing (TED ≥ 0)

Requirement: the built weapon must have **zero overswing**, i.e. TED ≥ 0.

$$\text{TED} \geq 0 \iff W\left(1 - \frac{E_{\text{eff}}}{100}\right) \leq 3 \iff W \leq \frac{300}{100 - E_{\text{eff}}}$$



How the solver enforces it:

- Up to 3 kg a build can never overswing, since $E_{\text{eff}} \geq 0$.
- The penalty $q(W)$ is concave above 3 kg, so on a weight interval its secant is a valid relaxation: MILP with TED ≥ 0 added.
- **Spatial branch and bound** on total weight: split at 3 kg, then bisect. A node result counts only if the real build has TED ≥ 0.
- Nodes are pruned once their bound cannot beat the incumbent. Defaults: 512 nodes, 60 s, tolerance 10^{-5}.

Equipment Ergonomics Modifier (b)

Real Combat Scenario: Body armor, armored rigs, helmets, and backpacks reduce your effective ergonomics.

Effective Ergonomics Loadout Formula

$$E_{\text{eff}} = E_{\text{weapon}} \cdot (1 + b), \quad b \in [-0.40,\ 0.00]$$

- b: Total equipment ergo penalty percentage from player loadout.
- Example: A heavy Tier-6 Zabralo armor ($b = -28\%$) + Altyn helmet ($b = -7\%$) $\implies$ $b = -35\%$.

Player Benefit: The optimizer automatically adjusts the overswing threshold to account for your armor, guaranteeing that the build snaps with zero sway in full raid gear.

Gunsmith Task Solver

Problem: Gunsmith quests require building weapons to exact specifications.

Task Definition (JSON):

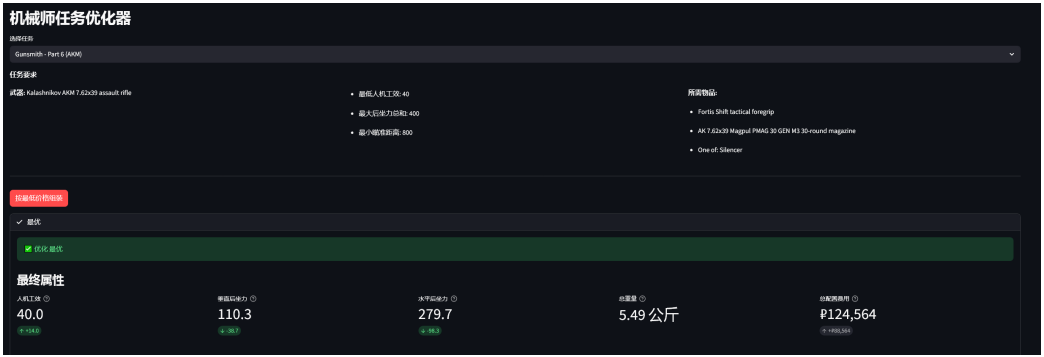
- Target weapon name
- Stat constraints:
  - Min ergonomics, max recoil
  - Min magazine capacity
  - Max weight, min sighting range
- Required specific items
- Required categories (e.g., "Silencer")

How It Works:

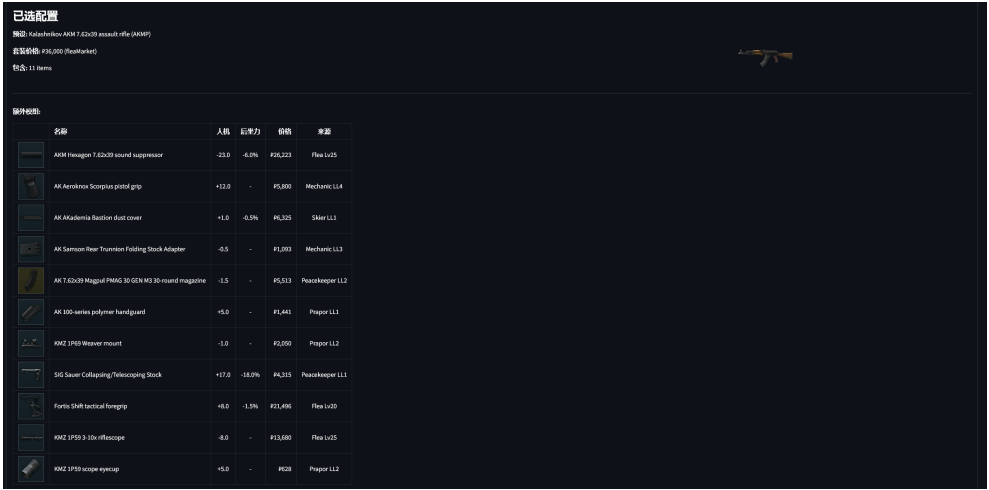
1. Load task from `tasks.json`
2. Convert requirements to constraints:
   - Required items $\rightarrow X_i = 1$
   - Categories $\rightarrow \sum_{i \in \text{cat}} X_i \geq 1$
   - Stats $\rightarrow$ hard bounds
3. Objective: minimize price
4. Solver finds cheapest valid build

Result: Automatically solves all 25 Gunsmith quests with optimal (cheapest) builds.

Gunsmith Task List

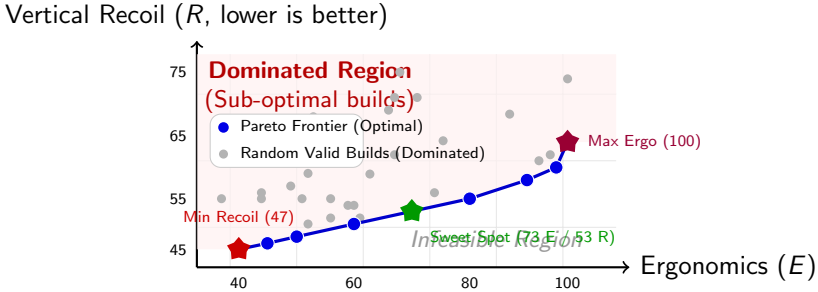


Gunsmith Solution Details



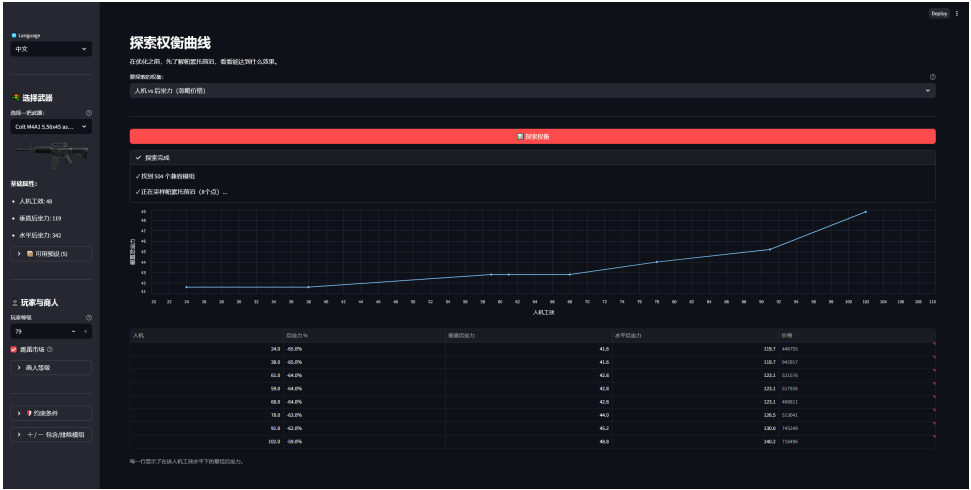
Pareto Frontier Analysis: Real Weapon Benchmark (M4A1)

Core Concept: The Pareto Frontier represents the boundary of non-dominated builds where no stat can be improved without sacrificing another.



Takeaway: Points inside the cloud represent wasted performance (e.g. higher recoil for the same ergonomics). The optimizer mathematically guarantees you stay on the blue boundary.

Pareto Frontier Visualization



Future Work & Unimplemented Features

- **Hidden Weapon Stats**: Currently unable to retrieve hidden weapon stats, such as recoil angle.
- **Trader Barters**: Currently unable to retrieve trader barter items and their calculated prices.

Acknowledgements

Data Source

TARKOV.DEV

Open-source GraphQL API for Escape from Tarkov

AI Coding Assistants



Claude Code

Anthropic



Gemini CLI

Google



MiniMax

MiniMax

All the AI assistants helped me with code generation, debugging, and documentation